

เอกสารประกอบการสอน

รายวิชา ENGSE206 การกำหนดความต้องการและการออกแบบทางซอฟต์แวร์

Software Requirements Specification and Design

สัปดาห์ที่ 1 | หน่วยที่ 1: ฐานคิดของวิศวกรรมความต้องการและความเข้าใจปัญหา

ระยะเวลา 5 ชั่วโมง: ทฤษฎี 2 ชั่วโมง และปฏิบัติ/Workshop 3 ชั่วโมง | แกนผลลัพธ์: CLO1

(Understand/Analyze)

ข้อบทรียน

- 1.1 บทบาทของความต้องการและการออกแบบในวงจรการพัฒนาซอฟต์แวร์
- 1.2 Problem, Requirement, Specification, Design และ Implementation: แนวคิดที่ต้องแยกให้ออก
- 1.3 กรณศึกษาแกนกลางของรายวิชา การทำงานเป็นทีม และการเริ่มต้น Problem Brief v0.1

วัตถุประสงค์การสอน

- 1.1.1 อธิบายเหตุผลที่โครงการซอฟต์แวร์ต้องเริ่มจากการทำความเข้าใจปัญหา ผู้ใช้ และความต้องการก่อนตัดสินใจออกแบบหรือเขียนโปรแกรมได้ (Understand)
- 1.1.2 อธิบายความสัมพันธ์ระหว่าง requirements, design, implementation, testing และเอกสาร/หลักฐานสำคัญใน SDLC ได้ (Understand)
- 1.2.1 จำแนกความแตกต่างระหว่าง problem, need, requirement, specification, design และ implementation จากสถานการณ์ตัวอย่างได้ (Analyze)
- 1.2.2 วิเคราะห์ข้อความปัญหาเบื้องต้นเพื่อระบุผู้ใช้ คุณค่าที่ต้องการ ขอบเขต และข้อสมมติที่ควรถามต่อได้ (Analyze)
- 1.3.1 ร่วมจัดทำ Problem Brief v0.1 ของกรณีศึกษาด้วยข้อมูลที่ตรวจสอบได้ และแยกข้อเท็จจริงออกจากข้อสมมติได้ (Apply)
- 1.3.2 ใช้พื้นที่ทำงานร่วมกันบนเบราร์เซออร์ จัดเก็บไฟล์ ตั้งชื่อเวอร์ชัน และส่งงานทีม/งานรายบุคคลตามข้อตกลงของรายวิชาได้ (Apply)

หน่วยที่ 1 ฐานคิดของวิศวกรรมความต้องการและความเข้าใจปัญหา

บทนำ

การพัฒนาซอฟต์แวร์ที่มีคุณภาพไม่เริ่มจากการเลือกภาษาโปรแกรมหรือน้ำจืดที่สวยงาม แต่เริ่มจากการทำความเข้าใจว่าใครกำลังเผชิญปัญหาอะไร เหตุใดปัญหานั้นจึงสำคัญ และระบบต้องช่วยให้ผู้ใช้บรรลุเป้าหมายอย่างไร รายวิชา ENGSE206 จึงทำหน้าที่เป็นสะพานเชื่อมจากทักษะการเขียนโปรแกรมและกระบวนการซอฟต์แวร์ในรายวิชาก่อนหน้า ไปสู่การวิเคราะห์ ออกแบบ พัฒนา และทำโครงการอย่างเป็นระบบในชั้นปีถัดไป

สัปดาห์แรกมุ่งสร้าง “ภาษากลาง” ของชั้นเรียนให้แยกความต่างของปัญหา ความต้องการ เอกสารข้อกำหนด การออกแบบ และการพัฒนาได้ชัดเจน พร้อมเริ่มต้นกรณีศึกษาเดียวที่ใช้ต่อเนื่องทั้งภาคการศึกษา

นักศึกษาจะทำงานเป็นทีมแบบวิศวกรรมซอฟต์แวร์ แต่มีหลักฐานรายบุคคลเพื่อยืนยันการเรียนรู้ของแต่ละคน

1. ความสอดคล้องผลลัพธ์การเรียนรู้ของสัปดาห์ที่ 1

รายการ	รายละเอียด
CLO หลัก	CLO1: อธิบายและวิเคราะห์บทบาทของวิศวกรรมความต้องการและการออกแบบซอฟต์แวร์ในวงจรการพัฒนาซอฟต์แวร์ รวมถึงผู้มีส่วนได้ส่วนเสีย ขอบเขต ข้อจำกัด และประเด็นจริยธรรม
PLO/Sub-PLO ที่สนับสนุน	PLO1: 1C, 1F และ PLO2: 2A (เน้นความเข้าใจหลักการ การอธิบายความต้องการ และการตีความสารสนเทศเพื่อแก้ปัญหา)
Bloom's Taxonomy	Understand และ Analyze; ช่วงปฏิบัติเสริม Apply ผ่านการจัดทำ Problem Brief v0.1
หลักฐานการเรียนรู้	Problem Brief v0.1 ของทีม (formative) และ Exit Ticket รายบุคคล (A1: 2% ของคะแนนรวมรายวิชา)

2. แผนเวลาและลำดับกิจกรรม 2 ชั่วโมงทฤษฎี + 3 ชั่วโมงปฏิบัติ

ช่วง	เวลา	สาระ/กิจกรรม	ผลลัพธ์ระหว่างเรียน
ทฤษฎี 1	0:00–0:25	กิจกรรมเปิดชั้นเรียน: “เขียนโค้ดได้แล้ว ทำไมระบบยังไม่ตอบโจทย์?” และอธิบายบทบาท requirements/design ใน SDLC	นักศึกษาระบุผลเสียของการข้าม ขั้นทำความเข้าใจผู้ใช้ได้
ทฤษฎี 2	0:25–1:05	บรรยายเชิงโต้ตอบ: Problem → Need → Requirement → Specification → Design → Implementation → Test	แยกแนวคิดและยกตัวอย่างได้
ทฤษฎี 3	1:05–2:00	อภิปรายกรณีศึกษาแกนกลางและสาริตหลักฐาน/artefacts ของรายวิชา	เห็นภาพชิ้นงานตลอดภาค และเตรียมเข้าสู่ Workshop
ปฏิบัติ 1	0:00–0:35	เตรียมพื้นที่ทำงานร่วมกันบนเบร้าวเซอร์ จัดทีม กำหนดบทบาท และอ่านโจทย์	ทีมพร้อมทำงาน มีโฟลเดอร์และข้อตกลงการทำงาน
ปฏิบัติ 2	0:35–1:55	Workshop: วิเคราะห์ปัญหา ผู้ใช้ เป้าหมาย ความเจ็บปวด และคำถามที่ยังไม่รู้	ร่าง Problem Brief v0.1
ปฏิบัติ 3	1:55–3:00	Mini pitch, peer questions, ปรับแก้ ส่งงานทีม และ Exit Ticket รายบุคคล	Problem Brief v0.1 + หลักฐานรายบุคคล

ประเด็นสำคัญของสัปดาห์นี้

“Requirement ไม่ใช่สิ่งที่ทีมเดาเอง” แต่เป็นความต้องการที่ผ่านการทำความเข้าใจผู้ใช้ ผู้มีส่วนได้ส่วนเสีย เป้าหมาย
ข้อจำกัด และหลักฐานที่ตรวจสอบได้ ก่อนแปลงเป็นเอกสารและแบบออกแบบ

3. เนื้อหาบรรยาย

3.1 บทเรียน 1.1: บทบาทของความต้องการและการออกแบบใน SDLC

งานพัฒนาซอฟต์แวร์สามารถมองเป็นสายธารของการตัดสินใจ ไม่ว่าจะใช้ Agile, iterative, incremental หรือแนวทางอื่น ทีมยังต้องตอบคำถามพื้นฐานให้ได้ว่า “กำลังแก้ปัญหาอะไรให้ใคร” ก่อนจึงจะเลือกได้ว่า “ควรออกแบบและพัฒนาอย่างไร” Requirements engineering ทำให้ทีมมีหลักฐานของความเข้าใจร่วมกัน ส่วน software design แปลงความเข้าใจนั้นเป็นแนวทางการสร้างระบบที่เป็นไปได้ ใช้งานได้ และดูแลรักษาได้

ช่วงของงาน	คำถามที่ต้องตอบ	ตัวอย่างหลักฐาน/artefact
Problem & Context	ใครกำลังมีปัญหา อะไรคือผลกระทบ และเหตุใดจึงควรแก้ตอนนี้	Problem brief, context note, stakeholder map
Requirements	ระบบหรือบริการต้องช่วยผู้ใช้อะไร ภายใต้ข้อจำกัดใด	Requirement backlog, user story/use case, non-functional requirements, SRS
Design	จะแบ่งส่วนของระบบ จัดข้อมูล สร้างการโต้ตอบ และวางทางเลือกเทคนิคอย่างไร	Design strategy, conceptual architecture, user flow, wireframe, detailed design
Implementation & Test	จะสร้าง ตรวจสอบ และส่งมอบสิ่งที่ออกแบบไว้ได้อย่างไร	Source code, test cases, test report, release note
Feedback & Change	สิ่งที่ส่งมอบคุณยังตอบโจทย์จริงหรือไม่ และต้องปรับอะไร	Review record, usability evidence, change request, traceability

ในรายวิชา ENGSE206 นักศึกษาจะไม่ถูกคาดหวังให้พัฒนาโปรแกรมสมบูรณ์ทุกสัปดาห์ แต่จะสร้าง Requirement-to-Design Package ซึ่งเป็นชุดหลักฐานที่สามารถส่งต่อให้ผู้พัฒนา ผู้ทดสอบ ผู้จัดการโครงการ หรือผู้เกี่ยวข้องในองค์กรนำไปใช้ได้ ชุดเอกสารนี้จะมีคุณค่าก็ต่อเมื่อเชื่อมโยงกันอย่างมีเหตุผล ตั้งแต่ปัญหา ความต้องการ การออกแบบ ไปจนถึงแนวคิดการทดสอบและการประเมิน

3.2 บทเรียน 1.2: แยกแนวคิดให้ชัดก่อนเริ่มวิเคราะห์

คำที่ดูคล้ายกันอาจทำให้มีสื่อสารคลาดเคลื่อน เช่น การพูดว่า “ต้องทำแอปจองห้อง”

อาจเป็นเพียงแนวทางออกแบบ ไม่ใช่ความต้องการที่แท้จริง หากทีมรับมือด้วยเทคโนโลยีหรือนำจอจะเสี่ยงต่อการพัฒนาสิ่งที่ไม่แก้ปัญหา

การแยกคำสำคัญต่อไปนี้จึงเป็นทักษะพื้นฐานของนักวิเคราะห์และนักออกแบบระบบ

คำสำคัญ	ความหมายในการทำงาน	ตัวอย่างจากระบบจองทรัพยากร
Problem	สถานการณ์หรือผลกระทบที่ไม่พึงประสงค์ ซึ่งควรมีหลักฐานสนับสนุน	เจ้าหน้าที่ตรวจสอบตารางจองผ่านหลายไฟล์ ทำให้เกิดการจองซ้ำและตอบผู้ใช้ล่าช้า
Need	ผลลัพธ์หรือคุณค่าที่ผู้ใช้/องค์กรต้องการ ไม่ผูกกับวิธีแก้	ผู้ใช้ต้องตรวจสอบช่วงเวลาว่างได้อย่างรวดเร็ว และทราบผลอนุมัติอย่างชัดเจน
Requirement	สิ่งที่ระบบหรือกระบวนการต้องทำ/ต้องมี เพื่อสนอง need ภายใต้ขอบเขตและข้อจำกัด	ระบบต้องแสดงสถานะห้องว่างตามวันที่ เวลา และประเภททรัพยากรให้ผู้ใช้ที่มีสิทธิ์เข้าดูได้
Specification	การเขียน requirement ให้ละเอียด ชัดเจน ตรวจสอบและทดสอบได้	FR-01: ผู้ใช้ที่ยืนยันตัวตนแล้วสามารถค้นหาห้องว่างตามวันที่และช่วงเวลาได้ โดยระบบต้องแสดงผลภายใน ... วินาที ภายใต้เงื่อนไขที่กำหนด
Design	การตัดสินใจว่า “จะสร้างอย่างไร” เพื่อรองรับ requirement	ออกแบบหน้า Search Availability, Booking Service, ตาราง Reservation และการเชื่อมต่อ Calendar
Implementation	การลงมือสร้างระบบจริงตามแบบออกแบบ	พัฒนา front-end, API, database migration และ automated tests

คำถามชวนคิด

ประโยค “ต้องทำแอปมือถือเพื่อจองห้อง” เป็น requirement หรือ design? คำตอบคือ ยังไม่เพียงพอที่จะตัดสินใจ ต้องกลับไปถามก่อนว่าใครเป็นผู้ใช้ เป้าหมายคืออะไร ใช้ในบริบทใด และมีเหตุผลใดที่อุปกรณ์เคลื่อนที่จึงจำเป็นหรือเหมาะสม

3.3 บทเรียน 1.3: กรณีศึกษาแกนกลางและสายธารชิ้นงานตลอดภาค

เพื่อให้เห็นการเชื่อมโยงของชิ้นงานอย่างเป็นรูปธรรม ชั้นเรียนจะใช้กรณีศึกษาแกนกลาง

“ระบบจัดการการจองห้องและอุปกรณ์การเรียนรู้ (Campus Resource Booking)”

กรณีนี้มีผู้เกี่ยวข้องหลายกลุ่ม มีข้อมูลตารางเวลาและสิทธิ์การใช้งาน มีขั้นตอนอนุมัติ

และมีประเด็นด้านความเป็นส่วนตัว จึงเหมาะกับการฝึกเก็บความต้องการ วิเคราะห์ความขัดแย้ง จัดทำ SRS

และต่อยอดเป็นการออกแบบระดับเริ่มต้น

องค์ประกอบกรณีศึกษา	สาระที่นักศึกษาต้องพิจารณาในสัปดาห์ที่ 1
ผู้เกี่ยวข้องหลัก	นักศึกษา/ผู้ใช้, อาจารย์ผู้สอน, เจ้าหน้าที่ห้องปฏิบัติการ, ผู้อนุมัติ, ผู้ดูแลระบบ, หน่วย IT
ปัญหาปัจจุบัน	การเช็คตารางจากหลายช่องทาง, การจองซ้ำ, สถานะไม่ชัดเจน, หลักฐานการอนุมัติไม่ครบ, รายงานการใช้งานจัดทำยาก
คุณค่าที่คาดหวัง	เห็นความพร้อมใช้ของทรัพยากร, ลดงานซ้ำของเจ้าหน้าที่, มีการติดตามสถานะ, มีข้อมูลเพื่อวางแผนใช้ทรัพยากร
ข้อควรระวัง	ไม่ใช่ทุกปัญหาต้องแก้ด้วยฟังก์ชันใหม่; ต้องแยก “ข้อเท็จจริง” “ความต้องการ” “ข้อสมมติ” และ “คำถามเปิด”
ขอบเขตการฝึก	เน้นบริการดิจิทัลระดับคณะ/หลักสูตร; ไม่ใช่ข้อมูลส่วนบุคคลจริง และไม่ต้องเชื่อมต่อบริบบนมหาวิทยาลัยจริง

3.4 การเชื่อมโยงกับรายวิชาอื่นในหลักสูตร

ช่วงการเรียนรู้	สิ่งที่นักศึกษามี/จะต่อยอด	บทบาทของ ENGSE206
ก่อนเรียน	ENGSE203: front-end/back-end, testing, tools, version control; ENGSE204: OOP; ENGSE205: process & quality; ENGSE219: database	ใช้ความรู้เทคนิคเป็นฐานในการถามคำถามที่สมเหตุสมผลและออกแบบสิ่งที่พัฒนาได้จริง
รายวิชานี้	Requirements engineering, SRS, traceability, conceptual design, UX/UI และ design evaluation	ทำให้การพัฒนาไม่เริ่มจากการเดา แต่มีหลักฐานและเหตุผลเชิงวิศวกรรม

ช่วงการเรียนรู้	สิ่งที่นักศึกษามี/จะต่อยอด	บทบาทของ ENGSE206
ต่อยอดชั้นปี 2-3	ENGSE207 Architecture, ENGSE208 System Analysis and Design, ENGSE210 Project Management, ENGSE212 Senior Project	นำเอกสารความต้องการและแนวคิดออกแบบไปใช้จัดการโครงการและพัฒนาระบบอย่างเป็นระบบ
ต่อยอดชั้นปี 4	สหกิจศึกษา/ฝึกประสบการณ์/ปัญหาพิเศษจากสถานประกอบการ	พร้อมสื่อสารกับผู้รับโจทย์จริง จัดทำเอกสาร และทบทวนการออกแบบในทีมวิชาชีพ

แนวปฏิบัติด้านจริยธรรมและการใช้ AI

ใช้ AI ได้เพื่อช่วยร่างคำถาม สรุบบันทึกข้อมูลที่ไม่ลับ หรือช่วยตรวจหาความกำกวม
แต่ห้ามส่งข้อมูลส่วนบุคคล/ข้อมูลลับเข้าสู่เครื่องมือ AI และต้องระบุว่าใช้ AI ช่วยอะไร
พร้อมตรวจสอบความถูกต้องกับหลักฐานจริงเสมอ

4. ปฏิบัติการ/Workshop สัปดาห์ที่ 1

ชื่อกิจกรรม: เริ่มต้นโครงงาน Requirement-to-Design Package และจัดทำ Problem Brief

v0.1

รูปแบบ: งานกลุ่ม 4–5 คน โดยใช้กรณีศึกษาแกนกลางเดียวกันทั้งชั้นเรียน

เอกสารฉบับนี้เน้นให้ทีมเริ่มต้นจากการทำความเข้าใจปัญหา มิใช่รีบออกแบบหน้าจอหรือเลือกเทคโนโลยี

4.1 ผลลัพธ์ที่ต้องได้เมื่อจบ Workshop

- มีไฟล์โครงงานของทีมและพื้นที่เอกสารร่วมกันที่เข้าถึงได้ทั้ง iMac (macOS) และ notebook ส่วนตัว (Windows) ผ่านเว็บเบราว์เซอร์
- มีรายชื่อสมาชิก บทบาทเบื้องต้น และกติกาการทำงานของทีม
- มี Problem Brief v0.1 ความยาวประมาณ 1 หน้า ที่อธิบายปัญหา ผู้ใช้ ผลกระทบ คุณค่า ข้อเสนอแนะ และคำถามเปิด
- สมาชิกทุกคนส่ง Exit Ticket รายบุคคล เพื่อยืนยันความเข้าใจเรื่อง problem/requirement/design และการมีส่วนร่วมของตนเอง

4.2 เครื่องมือที่ใช้และแนวทางการทำงานข้ามระบบปฏิบัติการ

เพื่อให้ไม่เกิดปัญหาไฟล์คนละเวอร์ชันในห้อง iMac M1 และเครื่องส่วนตัว Windows ให้ใช้แนวทาง browser-first: ทำงานร่วมกันบน Google Drive/Google Docs และส่งงานผ่าน Google Classroom ของรายวิชา โดยเปิดผ่าน Chrome, Edge หรือ Safari ได้ ไม่ต้องติดตั้งโปรแกรมเฉพาะระบบปฏิบัติการในสัปดาห์แรก

งานที่ต้องทำ	Windows notebook	iMac M1 (macOS)	ข้อควรระวังร่วมกัน
เข้าสู่พื้นที่ทำงาน	เปิด Chrome/Edge → เข้าสู่ Google Classroom และ Google Drive	เปิด Safari/Chrome → เข้าสู่ Google Classroom และ Google Drive	ใช้บัญชีมหาวิทยาลัย/บัญชีที่ผู้สอนกำหนดเพียงบัญชีเดียวตลอดภาค
สร้าง/แก้ไขเอกสารร่วม	เปิด Google Docs จาก Drive; บันทึกอัตโนมัติ	เปิด Google Docs จาก Drive; บันทึกอัตโนมัติ	อย่าดาวน์โหลดไปแก้ไขคนละไฟล์เมื่อกำลังทำงานร่วมกัน

งานที่ต้องทำ	Windows notebook	iMac M1 (macOS)	ข้อควรระวังร่วมกัน
ทางลัดหลัก	Ctrl+L ท้ายเว็บ, Ctrl+C/V คัดลอก/วาง, Ctrl+P พิมพ์/บันทึก PDF	Command+L ท้ายเว็บ, Command+C/V คัดลอก/วาง, Command+P พิมพ์/บันทึก PDF	ใช้เมนู File → Download → PDF เมื่อเตรียมส่งงาน
จัดเก็บไฟล์	File Explorer ใช้ตรวจไฟล์ที่ดาวน์โหลดแล้ว	Finder ใช้ตรวจไฟล์ที่ดาวน์โหลดแล้ว	ชื่อไฟล์ต้องตามมาตรฐานเดียวกัน และห้ามแก้ไขไฟล์ PDF หลังส่ง

มาตรฐานไฟล์เดอร์ของทีม

สร้างไฟล์เดอร์ชื่อ ENGSE206_2569_TeamXX แล้วมี 01-Brief, 02-Evidence, 03-Requirements, 04-Design, 99-Submission และ Team_Worklog. ผู้สอนสามารถตรวจ revision history เพื่อดูการทำงานร่วมกันได้

4.3 ขั้นตอน Workshop (3 ชั่วโมง)

เวลา	ขั้นตอน	บทบาทของผู้เรียน	หลักฐาน/จุดตรวจ
0:00–0:15	เข้าห้องเรียนออนไลน์/Drive, สร้างไฟล์เดอร์ และเปิดไฟล์แม่แบบ	สมาชิกทุกคนเข้าสู่ shared folder; ผู้ประสานทีมแชร์สิทธิ์ให้ครบ	ผู้สอนตรวจชื่อทีมและส มาชิกครบ
0:15–0:35	กำหนดบทบาทแบบหมุนเวียน: facilitator, note taker, evidence checker, timekeeper, presenter	ตกลงกติกาการคุย การอ้างอิงแหล่งข้อมูล และการใช้ AI	Team working agreement 5–7 ข้อ
0:35–1:05	อ่านกรณีศึกษาและแยกข้อมูลเป็น Facts / Assumptions / Open Questions	ช่วยกันวงข้อความที่เป็นข้อมูลจริง ข้อสมมติ และคำถามที่ต้องไปเก็บเพิ่ม	กระดาน/ตาราง Facts–Assumptions– Questions
1:05–1:35	ระบุผู้ใช้หลัก ปัญหา ผลกระทบ และคุณค่าที่ต้องการ	ไข่มุมมองผู้ใช้ ไม่เริ่มจากชื่อฟีเจอร์หรือเทคโนโลยี	ร่างส่วน Problem, Users, Value
1:35–2:05	เขียน Problem Brief v0.1 ตามแม่แบบ	ผู้บันทึกเขียนร่วมกับทีม; evidence checker ตรวจสอบว่ามีข้อสมมติชัดเจน	Problem Brief v0.1 draft

เวลา	ขั้นตอน	บทบาทของผู้เรียน	หลักฐาน/ตรวจสอบ
2:05–2:30	Mini pitch ทีมละ 2 นาที + peer questions	ผู้นำเสนออธิบายปัญหา; สมาชิกอื่นจดคำถามที่ช่วยให้ชัดเจน	รายการคำถาม/ข้อเสนอแนะจากเพื่อน
2:30–2:45	ปรับแก้และตรวจ checklist ก่อนส่ง	ทีมตรวจข้อไฟล์ สมาชิก วันที่ เวอร์ชัน และ AI disclosure	ไฟล์ PDF + ลิงก์ editable source
2:45–3:00	ส่งงานทีมและทำ Exit Ticket รายบุคคล	ทุกคนส่งรหัส/คำตอบของตนเอง	หลักฐานการส่งครบทั้งทีมและรายบุคคล

4.4 โครงสร้าง Problem Brief v0.1

หัวข้อ	คำแนะนำในการเขียน
1. ชื่อทีมและชื่อกรณีศึกษา	ระบุ Team XX, รายชื่อสมาชิกและวันที่จัดทำ; ใช้ชื่อระบบชั่วคราวได้
2. สถานการณ์ปัจจุบัน	อธิบายวิธีการทำงานเดิมและอาการของปัญหา โดยไม่ใช่คำตอบเชิงเทคนิคก่อนเวลา
3. ผู้ใช้/ผู้เกี่ยวข้องที่ได้รับผลกระทบ	ระบุอย่างน้อย 3 กลุ่ม พร้อมบทบาทและสิ่งที่แต่ละกลุ่มต้องการ
4. ปัญหาและผลกระทบ	เขียนให้เห็นผลต่อเวลา คุณภาพ ความผิดพลาด การสื่อสาร หรือประสบการณ์ผู้ใช้
5. คุณค่าที่คาดหวัง	บอกผลลัพธ์ที่ควรดีขึ้นเมื่อแก้ปัญหา ไม่ใช่รายชื่อเมนูหรือหน้าจอ
6. ขอบเขตเริ่มต้น	ระบุ In scope / Out of scope แบบกว้าง ๆ เพื่อป้องกันงานบานปลาย
7. Facts / Assumptions / Open Questions	ทำให้ทีมรู้ว่าอะไรยืนยันแล้ว อะไรเป็นข้อสมมติ และต้องถามใครต่อในสัปดาห์หน้า
8. การใช้ AI (ถ้ามี)	ระบุเครื่องมือ สิ่งที่ได้ช่วย และการตรวจสอบ/แก้ไขโดยทีม

5. งานที่มอบหมาย วิธีส่งงาน และการวัดผล

5.1 งานที่มอบหมาย

ชิ้นงาน	ลักษณะงาน	ผู้ส่ง	สถานะคะแนน	กำหนดส่ง
W01-Group: Problem Brief v0.1	งานกลุ่ม 4-5 คน; เขียนตามแม่แบบ 1 หน้า พร้อม Facts/Assumptions/Open Questions และ AI disclosure	หัวหน้าทีมส่ง 1 ครั้ง; สมาชิกทุกคนตรวจสอบชื่อและเข้ากดส่ง/รับทราบใน Classroom	Formative 0%; เป็นจุดตั้งต้นของ Requirements Engineering Portfolio (A3: 25%)	ภายใน 23:59 ของวันเรียน หรือเวลาที่ผู้สอนประกาศใน LMS
W01-Individual: Exit Ticket	งานรายบุคคล 3 ข้อ: อธิบายความเสี่ยงจากการเริ่มเขียนโค้ดก่อนเข้าใจ requirement; จำแนกคำสำคัญ; ตั้งคำถามต่อ stakeholder 1 ข้อ	สมาชิกทุกคนส่งด้วยตนเองผ่าน Google Form/Classroom	A1 = 2% ของคะแนนรวมรายวิชา (ส่วนหนึ่งของ Learning Check 10%)	ก่อนออกจากห้องปฏิบัติการ หรือเวลาที่ผู้สอนประกาศ
Team Worklog	บันทึกบทบาท งานที่ทำ เวลา และผู้รับผิดชอบของสมาชิกแต่ละคน	ทีมอัปเดตใน shared folder	ไม่คิดคะแนนแยกในสัปดาห์นี้; ใช้เป็นหลักฐานประกอบ A5 (peer review/reflection 5%)	อัปเดตพร้อมส่งงานกลุ่ม

5.2 ขั้นตอนการส่งงาน (Google Classroom + Google Drive)

1. เปิดหัวข้องาน “ENGSE206 / W01 / Problem Brief v0.1” ใน Google Classroom และดาวน์โหลดไฟล์แม่แบบ หรือคัดลอกไฟล์เข้าโฟลเดอร์ของทีมใน Google Drive
2. ทีมทำงานในเอกสารเดียวกันบน Google Docs โดยตั้งชื่อไฟล์ ENGSE206_TeamXX_W01_ProblemBrief_v0.1
3. ก่อนส่ง เลือก File → Download → PDF Document (.pdf) เพื่อสร้างไฟล์ส่ง; ตรวจสอบว่า PDF อ่านได้ ชื่อสมาชิกครบ และมีเลขเวอร์ชัน v0.1
4. หัวหน้าทีมอัปโหลด PDF ไปยัง Google Classroom พร้อมแนบลิงก์เอกสารต้นฉบับที่ตั้งสิทธิ์ “ผู้สอนดูหรือแสดงความคิดเห็นได้”; สมาชิกทุกคนเปิดงานและกด “ส่งงาน/Mark as done” ตามขั้นตอนที่ผู้สอนกำหนด
5. สมาชิกแต่ละคนทำ Exit Ticket ด้วยบัญชีของตนเอง ห้ามให้เพื่อนตอบแทน เพราะเป็นหลักฐานรายบุคคล
6. หากเกิดปัญหาอินเทอร์เน็ต ให้บันทึกไฟล์ไว้ในเครื่องชั่วคราวแล้วส่งภายในเวลาที่ผู้สอนประกาศ พร้อมแจ้งเหตุผลผ่าน Classroom; ห้ามส่งไฟล์ผ่านแชตส่วนตัวแทนระบบโดยไม่มีหลักฐานเวลา

มาตรฐานชื่อไฟล์

ENGSE206_TeamXX_W01_ProblemBrief_v0.1.pdf | ห้ามใช้ชื่อไฟล์ เช่น final, finalจริง, งานใหม่, สำเนา 2 เพราะทำให้ติดตามเวอร์ชันและการให้ feedback ยาก

5.3 เกณฑ์การวัดผลสัปดาห์ที่ 1

รายการประเมิน	เกณฑ์	คะแนน
Exit Ticket รายบุคคล (A1)	อธิบายความเชื่อมโยงระหว่าง problem–requirement–design ได้ถูกต้อง; จำแนกตัวอย่างได้; ตั้งคำถามที่ช่วยลดความไม่แน่นอนได้	2% ของคะแนนรวมรายวิชา
Problem Brief v0.1 ของทีม	ตรวจสอบความครบถ้วนขององค์ประกอบ การแยก facts/assumptions/open questions การตั้งชื่อและการส่งตามมาตรฐาน; ให้ feedback เพื่อปรับในสัปดาห์ 2	Formative 0%
การมีส่วนร่วมระหว่าง Workshop	ตรวจจาก worklog, revision history, การตอบคำถาม และ peer observation; เก็บเป็นหลักฐานประกอบความรับผิดชอบรายบุคคล	Feedback / หลักฐานสะสม

แนวให้คะแนน Exit Ticket แบบย่อ: 2 คะแนน = ครบถ้วนและมีเหตุผล; 1 คะแนน = เข้าใจบางส่วนแต่ยังสับสน; 0 คะแนน = ไม่ส่ง/ตอบไม่สะท้อนความเข้าใจ โดยคะแนนนี้อยู่ในกิจกรรม A1 ตามแผนประเมินผลรายวิชา ไม่ใช่คะแนนโครงงานกลุ่ม

ใบบันทึกการสอน

ใช้บันทึกหลังการสอนจริง เพื่อปรับปรุงกิจกรรม สื่อ เวลา และการช่วยเหลือนักศึกษาในครั้งถัดไป

วิธีสอนและกิจกรรม	1) ตั้งคำถามนำเข้าสู่บทเรียนจากประสบการณ์การพัฒนาโปรแกรมของนักศึกษา 2) บรรยายเชิงโต้ตอบ พร้อมตัวอย่าง Problem → Requirement → Specification → Design → Implementation 3) Think-Pair-Share: จำแนกข้อความตัวอย่างว่าเป็น problem / need / requirement / design 4) Case discussion: วิเคราะห์กรณีระบบจัดการการจองห้องและอุปกรณ์ 5) Workshop กลุ่ม: สร้างโฟลเดอร์ร่วม กำหนดบทบาท และจัดทำ Problem Brief v0.1 6) Mini pitch + peer questions + revision + submission + exit ticket
สื่อการสอน	• สไลด์/เอกสารประกอบการสอนสัปดาห์ที่ 1 • กรณีสืบศึกษา Campus Resource Booking • แม่แบบ Problem Brief v0.1 และ Team Worklog • Google Classroom, Google Drive, Google Docs • เครื่อง iMac M1, notebook Windows ของนักศึกษา, web browser และอินเทอร์เน็ต
หนังสืออ้างอิง	1) Sommerville, I. (2016). Software Engineering (10th ed.). Pearson. 2) Wiegers, K. E., & Beatty, J. (2013). Software Requirements (3rd ed.). Microsoft Press. 3) ISO/IEC/IEEE 29148:2018. Systems and software engineering - Life cycle processes - Requirements engineering.
งานที่มอบหมาย	งานกลุ่ม: Problem Brief v0.1 (formative) ส่ง PDF + ลิงก์เอกสารต้นฉบับผ่าน Classroom งานรายบุคคล: Exit Ticket 3 ข้อ ผ่าน Google Form/Classroom หลักฐานที่ม: Team Worklog และ revision history

การวัดผล	- Exit Ticket รายบุคคล: 2% ของคะแนนรวมรายวิชา (A1) - Problem Brief v0.1: Formative feedback ไม่คิดคะแนนแยก - การสังเกต/Worklog/Revision history: หลักฐานสะสมสำหรับ peer review และ reflection
บันทึกการสอน	จำนวนนักศึกษาที่เข้าเรียน: คน จำนวนนักศึกษาที่ส่ง Exit Ticket: คน จำนวนทีมที่ส่ง Problem Brief v0.1: ทีม ประเด็นที่นักศึกษาสับสน/คำถามที่พบ: การปรับปรุงสำหรับสัปดาห์ที่ 2:

ภาคผนวก ก โครงสร้างแม่แบบ Problem Brief v0.1

นักศึกษาคัดลอกแม่แบบ Word ที่จัดเตรียมแยกให้เข้าสู่ Google Docs ของทีม แล้วพัฒนาเป็นเอกสารจริงระหว่าง Workshop เอกสารหลักต้องมีข้อมูลตามรายการต่อไปนี้

หัวข้อที่ต้องมี	เกณฑ์ความครบถ้วนขั้นต่ำ
ข้อมูลทีมและเวอร์ชัน	Team XX, รายชื่อ/บทบาทสมาชิก, วันที่ และ v0.1
สถานการณ์ปัจจุบัน	บอกวิธีทำงานเดิมและอาการปัญหา โดยไม่ข้ามไปเสนอเทคโนโลยี
ผู้ใช้/ผู้เกี่ยวข้อง	ระบุอย่างน้อย 3 กลุ่ม พร้อมเป้าหมายหรือผลกระทบของแต่ละกลุ่ม
Problem statement และผลกระทบ	สรุปสิ่งที่ต้องแก้ และเหตุที่ปัญหานี้มีความสำคัญ
คุณค่าที่คาดหวัง	อธิบายผลลัพธ์ที่ควรดีขึ้นเมื่อแก้ปัญหา ไม่ใช่รายชื่อฟีเจอร์
ขอบเขตเริ่มต้น	ระบุ In scope และ Out of scope แบบกว้าง ๆ
Facts / Assumptions / Open Questions	แยกข้อมูลที่ยืนยันแล้ว ข้อสมมติ และคำถามที่ต้องถามต่อ
AI usage disclosure	บันทึกเครื่องมือ สิ่งที่ใช้ช่วย และวิธีที่ทีมตรวจสอบความถูกต้อง

ไฟล์แม่แบบสำหรับ Workshop

ใช้ไฟล์ ENGSE206_Week01_ProblemBrief_v0.1_Template.docx เป็นไฟล์ตั้งต้นของทีม และส่งออกเป็น PDF ตามขั้นตอนการส่งงานในหัวข้อ 5.2

ภาคผนวก ข Exit Ticket รายบุคคล

ส่งผ่าน Google Form/Classroom โดยใช้คำตอบของตนเอง ความยาวประมาณ 100–180 คำรวมกัน

1. ยกตัวอย่างความเสี่ยง 1 ประการที่อาจเกิดขึ้นเมื่อทีมเริ่มเขียนโปรแกรมก่อนเข้าใจปัญหาและความต้องการของผู้ใช้ พร้อมอธิบายเหตุผลสั้น ๆ

.....

.....

2. จากข้อความ “เจ้าหน้าที่ต้องเห็นสถานะคำขอใช้ห้องแบบทันเวลา” ให้ระบุว่าเป็น problem, need, requirement, specification หรือ design และอธิบายว่าทำไม

.....

.....

3. ระบุ stakeholder 1 กลุ่มของกรณีศึกษา และเขียนคำถาม 1 ข้อที่คุณอยากถามเพื่อทำความเข้าใจงานของกลุ่มนั้นให้ชัดเจน

.....

.....

บรรณานุกรม

หลักสูตรวิศวกรรมศาสตรบัณฑิต สาขาวิชาวิศวกรรมซอฟต์แวร์ (หลักสูตรใหม่ พ.ศ. 2566).

มหาวิทยาลัยเทคโนโลยีราชมงคลธัญบุรี.

International Organization for Standardization. (2018). ISO/IEC/IEEE 29148:2018 Systems and software engineering - Life cycle processes - Requirements engineering.

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.

Wieggers, K. E., & Beatty, J. (2013). Software Requirements (3rd ed.). Microsoft Press.